

Django Interview Questions & Answers

Fresher / Backend Developer Interview Preparation Guide

Format: Question → simple interview-ready answer → code example where useful.

1. Django Fundamentals

Q1. What is Django?

Answer: Django is a high-level Python web framework used to build secure, scalable web applications quickly. It provides URL routing, ORM, authentication, admin, forms, templates and security features.

Q2. Why is Django popular?

Answer: It is written in Python, supports rapid development, has a powerful ORM and admin panel, includes authentication and security features, and has a large ecosystem.

Q3. What architecture does Django use?

Answer: Django commonly follows the MVT pattern: Model handles data, View handles request/business logic, and Template handles presentation.

Q4. What is MVT?

Answer: Model represents database data; View contains request-handling logic; Template is the presentation layer.

2. Project & App

Q5. What is a Django project?

Answer: The overall configuration and collection of applications that make up a Django website.

Q6. What is a Django app?

Answer: A reusable component that implements a particular feature, such as accounts, products, blog or payments.

Q7. Difference between project and app?

Answer: A project is the complete site configuration and can contain multiple apps. An app is a specific reusable feature.

Q8. What is manage.py?

Answer: A command-line utility used for Django tasks such as running the server, migrations, creating a superuser and opening the shell.

3. Settings

Q9. What is settings.py?

Answer: The main configuration file containing settings such as `INSTALLED_APPS`, `DATABASES`, `MIDDLEWARE`, `TEMPLATES`, `STATIC_URL`, `SECRET_KEY`, `DEBUG` and `ALLOWED_HOSTS`.

Q10. What is `INSTALLED_APPS`?

Answer: A list of Django and third-party/local applications enabled in the project.

Q11. What is `ALLOWED_HOSTS`?

Answer: It defines host/domain names that Django is allowed to serve. It is important for production security.

Q12. What is `DEBUG`?

Answer: A development setting that enables detailed error information. It should normally be `False` in production.

4. URLs & Views

Q13. What is URL routing?

Answer: It maps incoming URL patterns to Django views.

Q14. What is path()?

Answer: A function used to define URL patterns.

Q15. What is include()?

Answer: It lets the project's URL configuration delegate URL patterns to an app's `urls.py`.

Q16. How do you pass a parameter through a URL?

Answer: Use converters such as `<int:id>` or `<str:name>` in the URL pattern.

```
path("student/<int:id>/", views.student)
```

Q17. What is reverse()?

Answer: It generates a URL from a named URL pattern rather than hardcoding the URL.

Q18. What is a Django view?

Answer: A view receives a request and returns a response, often by querying models and rendering a template.

Q19. What is render()?

Answer: `render()` combines a template with context data and returns an HTTP response.

```
def home(request):  
    return render(request, "home.html", {"name": "Shivam"})
```

Q20. What is HttpResponseRedirect?

Answer: A simple way to return an HTTP response directly.

```
def home(request):  
    return HttpResponseRedirect("Hello Django")
```

Q21. What is redirect()?

Answer: It sends the browser to another URL.

```
return redirect("home")
```

5. Templates

Q22. What is a Django template?

Answer: An HTML file that can contain Django Template Language for displaying dynamic data and controlling presentation.

Q23. What is template inheritance?

Answer: It allows multiple templates to reuse a common base layout using `extends` and `block` tags.

```
{% extends "base.html" %}  
{% block content %}  
<h1>Home</h1>  
{% endblock %}
```

Q24. What are template tags?

Answer: Tags perform template operations such as conditions, loops, URL generation and block inheritance.

```
{% if user.is_authenticated %}  
    <p>Welcome</p>  
{% endif %}
```

Q25. What are template filters?

Answer: Filters transform displayed values, for example upper, lower and length.

```
{{ name|upper }}
```

6. Models & ORM

Q26. What is a Django model?

Answer: A Python class representing application data and normally mapping to a database table.

```
class Student(models.Model):
    name = models.CharField(max_length=100)
    age = models.IntegerField()
```

Q27. What is ORM?

Answer: Object-Relational Mapping lets you interact with a database through Python objects and QuerySets instead of writing SQL for every operation.

```
Student.objects.all()
```

Q28. What is objects?

Answer: The default model manager used to create and query database records.

Q29. How do you create an object?

Answer: Use Model.objects.create() or instantiate the model and call save().

```
Student.objects.create(name="Shivam", age=21)
```

Q30. How do you retrieve all objects?

Answer: Use Model.objects.all().

```
students = Student.objects.all()
```

Q31. How do you retrieve one object?

Answer: Use get() when exactly one matching object is expected.

```
student = Student.objects.get(id=1)
```

Q32. What is filter()??

Answer: It returns a QuerySet containing objects matching conditions.

```
students = Student.objects.filter(age=21)
```

Q33. get() vs filter()?

Answer: get() expects one result and can raise an exception if zero or multiple results exist. filter() returns a QuerySet and can contain zero or many results.

Q34. How do you update an object?

Answer: Change its fields and call save().

```
student = Student.objects.get(id=1)
student.age = 22
student.save()
```

Q35. How do you delete an object?

Answer: Call delete() on the object or a QuerySet.

```
student.delete()
```

7. Migrations

Q36. What is a migration?

Answer: A migration records database schema changes resulting from model changes.

Q37. makemigrations vs migrate?

Answer: makemigrations creates migration files from model changes; migrate applies those migrations to the database.

```
python manage.py makemigrations
python manage.py migrate
```

Q38. What happens after changing a model?

Answer: Usually run makemigrations and then migrate to apply the schema change.

8. Database Relationships

Q39. What is ForeignKey?

Answer: A many-to-one relationship. For example, many students can belong to one department.

```
department = models.ForeignKey(Department, on_delete=models.CASCADE)
```

Q40. What is OneToOneField?

Answer: A one-to-one relationship where one record corresponds to one related record.

```
user = models.OneToOneField(User, on_delete=models.CASCADE)
```

Q41. What is ManyToManyField?

Answer: A many-to-many relationship where each side can have multiple related records.

```
courses = models.ManyToManyField(Course)
```

Q42. What is on_delete=models.CASCADE?

Answer: It specifies that related records should be deleted when the referenced object is deleted.

9. Forms

Q43. What is Django Form?

Answer: A Form helps collect, validate and process user input.

```
class StudentForm(forms.Form):
    name = forms.CharField(max_length=100)
    age = forms.IntegerField()
```

Q44. What is ModelForm?

Answer: A ModelForm is a form generated from a model and is convenient for model-backed CRUD operations.

```
class StudentForm(forms.ModelForm):
    class Meta:
        model = Student
        fields = ["name", "age"]
```

Q45. Form vs ModelForm?

Answer: Form fields are defined manually and do not automatically map to a model. ModelForm derives fields from a model and supports saving model instances.

10. Authentication, Sessions & Cookies

Q46. Does Django provide authentication?

Answer: Yes. Django provides users, password hashing, login, logout, permissions and groups.

Q47. How do you authenticate a user?

Answer: Use django.contrib.auth.authenticate().

```
user = authenticate(username=username, password=password)
```

Q48. How do you log in a user?

Answer: Use django.contrib.auth.login(request, user).

```
login(request, user)
```

Q49. How do you log out?

Answer: Use `django.contrib.auth.logout(request)`.

```
logout(request)
```

Q50. How do you protect a view?

Answer: Use `login_required` for function-based views or `LoginRequiredMixin` for class-based views.

```
@login_required
def dashboard(request):
    return render(request, "dashboard.html")
```

Q51. What is a session?

Answer: A session stores user-specific state across requests. Django commonly stores session data server-side while the browser holds a session identifier.

```
request.session["username"] = "Shivam"
```

Q52. What is a cookie?

Answer: Small data stored in the browser and sent with requests to the relevant site.

```
response.set_cookie("theme", "dark")
```

11. CSRF

Q53. What is CSRF?

Answer: Cross-Site Request Forgery is an attack where a victim's browser is tricked into sending an unwanted authenticated request. Django provides CSRF protection for unsafe requests.

Q54. How do you use the CSRF token in a form?

Answer: Include `{% csrf_token %}` inside a POST form.

```
<form method="POST">
    {% csrf_token %}
    <input name="name">
    <button type="submit">Submit</button>
</form>
```

12. Middleware

Q55. What is middleware?

Answer: Middleware is a layer that processes requests before views and responses after views. It can implement application-wide behavior such as security, sessions and authentication.

Q56. Why is middleware used?

Answer: For security, sessions, authentication-related processing, logging, request/response processing and custom global behavior.

13. Static & Media

Q57. What are static files?

Answer: Files such as CSS, JavaScript and site images that are part of the application.

```
STATIC_URL = "static/"
```

Q58. What are media files?

Answer: Files uploaded by users, such as profile pictures and resumes.

```
MEDIA_URL = "media/"
MEDIA_ROOT = BASE_DIR / "media"
```

Q59. What is collectstatic?

Answer: A management command that gathers static files into the configured static root, commonly for production deployment.

```
python manage.py collectstatic
```

14. Django Admin

Q60. What is Django Admin?

Answer: A built-in administrative interface for managing model data.

Q61. How do you create a superuser?

Answer: Run createsuperuser and follow the prompts.

```
python manage.py createsuperuser
```

Q62. How do you register a model?

Answer: Import the model and register it with admin.site.

```
from django.contrib import admin
from .models import Student
admin.site.register(Student)
```

15. Class-Based Views

Q63. What is a CBV?

Answer: CBV means Class-Based View. It uses classes to organize request-handling behavior.

```
class HomeView(View):
    def get(self, request):
        return HttpResponse("Home")
```

Q64. FBV vs CBV?

Answer: Function-Based Views are simple and explicit. Class-Based Views support reusable behavior and work well with Django's generic views.

Q65. What are generic class-based views?

Answer: Reusable views for common tasks such as ListView, DetailView, CreateView, UpdateView, DeleteView and TemplateView.

16. QuerySets & Optimization

Q66. What is a QuerySet?

Answer: A QuerySet represents a collection of database queries/results managed by Django ORM.

```
students = Student.objects.all()
```

Q67. What is lazy evaluation?

Answer: A QuerySet generally delays executing its database query until its results are actually needed.

Q68. What is order_by()?

Answer: It orders query results.

```
Student.objects.all().order_by("-age")
```

Q69. What is exclude()?

Answer: It returns objects that do not match the supplied conditions.

```
Student.objects.exclude(age=18)
```

Q70. What is count()?

Answer: It returns the number of matching records.

```
Student.objects.count()
```

Q71. What is `select_related()`?

Answer: It is generally used for ForeignKey and OneToOne relationships and fetches related objects using SQL joins.

```
Student.objects.select_related("department")
```

Q72. What is `prefetch_related()`?

Answer: It is generally used for ManyToMany and reverse relationships and fetches related collections efficiently using additional queries.

```
Student.objects.prefetch_related("courses")
```

Q73. `select_related()` vs `prefetch_related()`?

Answer: `select_related` is mainly for single-valued relationships and uses joins; `prefetch_related` is mainly for collections and uses separate queries combined by Django.

17. Transactions & Signals

Q74. What is a database transaction?

Answer: A transaction groups operations so they can be committed together or rolled back if an error occurs.

```
from django.db import transaction

with transaction.atomic():
    student.save()
    profile.save()
```

Q75. What are Django signals?

Answer: Signals allow code to react to events such as model save/delete events.

```
from django.db.models.signals import post_save
from django.dispatch import receiver

@receiver(post_save, sender=Student)
def student_created(sender, instance, created, **kwargs):
    if created:
        print("Student created")
```

Q76. When should signals be used?

Answer: They can be useful for loosely coupled side effects, but should not be overused because they can make application flow harder to understand.

18. Django REST Framework

Q77. What is Django REST Framework?

Answer: DRF is a toolkit for building Web APIs with Django. It provides serializers, API views, ViewSets, routers, authentication and permissions.

Q78. What is a serializer?

Answer: A serializer converts model instances and other complex data into Python primitives suitable for JSON and validates incoming data.

```
class StudentSerializer(serializers.ModelSerializer):
    class Meta:
        model = Student
        fields = "__all__"
```

Q79. What is `APIView`?

Answer: `APIView` is a DRF class-based view that gives explicit control over HTTP methods.

```
class StudentAPI(APIView):
    def get(self, request):
        return Response({"message": "Hello"})
```

Q80. What is a ViewSet?

Answer: A ViewSet groups related API operations in one class and commonly works with routers.

```
class StudentViewSet(ModelViewSet):
    queryset = Student.objects.all()
    serializer_class = StudentSerializer
```

Q81. What is a router?

Answer: A router automatically generates URL patterns for registered ViewSets.

```
router = DefaultRouter()
router.register("students", StudentViewSet)
```

Q82. What is JSON?

Answer: JSON is JavaScript Object Notation, a common format for exchanging structured data between clients and APIs.

```
{"name": "Shivam", "age": 21}
```

19. API Authentication & Permissions

Q83. Authentication vs permission?

Answer: Authentication determines who the user is. Permission determines what that authenticated user is allowed to do.

Q84. What is JWT?

Answer: JWT stands for JSON Web Token. It is a token format commonly used for stateless API authentication.

20. Django Security

Q85. How does Django help secure applications?

Answer: Django provides security mechanisms for CSRF, SQL injection protection through normal ORM usage, XSS-related template escaping, clickjacking protection, secure password hashing and host-header validation.

Q86. Does Django protect against SQL injection?

Answer: Normal Django ORM operations use parameterized queries, helping protect against SQL injection. Avoid constructing SQL by concatenating untrusted input.

Q87. How does Django store passwords?

Answer: Django stores password hashes using its password-hashing framework rather than plain-text passwords.

21. Deployment

Q88. Can Django's development server be used in production?

Answer: No. It is intended for development. Production commonly uses a WSGI or ASGI application server behind a reverse proxy such as Nginx.

Q89. What is Gunicorn?

Answer: A Python WSGI HTTP server commonly used to serve Django applications in production.

Q90. What is Nginx?

Answer: A web server and reverse proxy commonly placed in front of Django to handle HTTP/HTTPS, static files and proxying.

Q91. What is ASGI?

Answer: ASGI stands for Asynchronous Server Gateway Interface. It supports asynchronous applications and protocols such as WebSockets.

Q92. WSGI vs ASGI?

Answer: WSGI is the traditional synchronous Python web interface. ASGI supports async applications and protocols such as WebSockets. Standard Django applications can run through WSGI; async/real-time features can benefit from ASGI.

22. Practical Interview Questions

Q93. How would you create a login system?

Answer: Collect credentials, validate the form, call `authenticate()`, call `login()` for a valid user, and redirect to the dashboard.

Q94. How would you create a CRUD application?

Answer: Create a model, then implement Create, Read, Update and Delete operations using views, forms/ModelForms, URLs, templates and the database.

Q95. How do you upload a file?

Answer: Use a `FileField/ImageField`, submit the form with `multipart/form-data`, and access uploaded files through `request.FILES`.

```
class Resume(models.Model):
    file = models.FileField(upload_to="resumes/")

# HTML form must use:
# enctype="multipart/form-data"
```

Q96. What is `request.FILES`?

Answer: It contains uploaded files received through a multipart form.

```
uploaded_file = request.FILES["file"]
```

Q97. How do you return JSON from Django?

Answer: Use `JsonResponse` in standard Django or `DRF Response` in Django REST Framework.

```
from django.http import JsonResponse

def api(request):
    return JsonResponse({"name": "Shivam"})
```

Q98. How do you handle GET and POST in one view?

Answer: Check `request.method` and process POST data when the method is POST.

```
def student(request):
    if request.method == "POST":
        name = request.POST.get("name")
    return render(request, "student.html")
```

Q99. What happens when a user enters a Django URL?

Answer: The request reaches Django, middleware processes it, the URL resolver finds a matching route, the view runs, it may access models and templates, and Django returns the response.

23. Top 50+ Questions to Revise Before a Fresher Interview

Priority list

Answer: Django basics; MVT; project vs app; `manage.py`; `settings.py`; `INSTALLED_APPS`; `ALLOWED_HOSTS`; `DEBUG`; URL routing; `path`; `include`; `views`; `render`; `templates`; `inheritance`; `models`; `ORM`; `QuerySets`; `get` vs `filter`; `create/save/delete`; `migrations`; `ForeignKey`; `OneToOneField`; `ManyToManyField`; `Forms`; `ModelForm`; `authentication`; `sessions`; `cookies`; `CSRF`; `middleware`; `static/media`; `admin`; `FBV` vs `CBV`; `generic views`; `select_related` vs `prefetch_related`; `transactions`; `signals`; `DRF`; `serializers`; `APIView`; `ViewSet`; `routers`; `authentication` vs `permissions`; `JWT`; `security`; `file uploads`; `CRUD`; `JSON`; `WSGI`; `ASGI`; `Gunicorn`; `Nginx`; `request-response lifecycle`.

Practical preparation

Answer: Be ready to build or explain a login system, CRUD app, file upload, REST API, database relationship, form validation and deployment flow. Practice explaining why you choose each Django component instead of only memorizing

syntax.